

# Repository Report: WoWAH Player Churn Prediction Pipeline

## What This Repository Is About

This repository implements a machine learning pipeline to predict **player churn** in *World of Warcraft* (WoW). Given a player and a month, the pipeline predicts whether that player will stop playing in the following month. The goal is to identify at-risk players one month in advance so that interventions can be applied proactively.

---

## The Dataset: World of Warcraft Avatar History (WoWAH)

The raw data comes from the **WoWAH** public dataset (Lee et al., ACM Multimedia Systems 2011). It covers **91,065 avatars** observed over **1,107 days** between January 2006 and January 2009. The data was collected by polling the game server every ~10 minutes, recording the state of every online character at that moment.

Each record (a “heartbeat”) contains:

| Field                         | Description                                                       |
|-------------------------------|-------------------------------------------------------------------|
| <code>timestamp</code>        | Date and time of the observation                                  |
| <code>char</code> (avatar ID) | Anonymized integer identifier for the character                   |
| <code>guild</code>            | Guild membership (-1 = no guild, otherwise an anonymized integer) |
| <code>level</code>            | Character level at time of observation                            |
| <code>race</code>             | Character race (e.g., Orc, Human, Night Elf, ...)                 |
| <code>charclass</code>        | Character class (e.g., Warrior, Mage, Priest, ...)                |
| <code>zone</code>             | In-game zone/location                                             |

Avatar and guild names were anonymized to positive integers for privacy. The original raw files are structured as `.txt` logs organized by quarter and day (e.g., `WoWAH/2006_01_03/2006-01-01/00-03-56.txt`), with one file per 10-minute polling interval.

**Data Files in `data/`**

| File                                | Size   | Description                                                                                                 |
|-------------------------------------|--------|-------------------------------------------------------------------------------------------------------------|
| wowah.rar                           | 551 MB | Original compressed raw log archive                                                                         |
| WoWAH/                              | —      | Extracted raw <code>.txt</code> log files, organized by quarter $\rightarrow$ day $\rightarrow$ time-of-day |
| wowah_parsed_all.csv                | 1.9 GB | Parsed CSV produced by <code>parser_mp.py</code> from the raw logs                                          |
| wowah_parsed_all.parquet            | 162 MB | Parquet version of the full parsed CSV                                                                      |
| wowah_data_raw.parquet              | 75 MB  | Cleaned raw data used as the historical input for feature engineering                                       |
| wowah_data_all.parquet              | 138 MB | Extended version of the raw data                                                                            |
| wowah_churn_data.parquet            | 385 KB | One row per (character, month) with a boolean <b>has_played</b> target column                               |
| wowah_user_month_with_churn.parquet | 41 MB  | Intermediate user-month dataset with churn labels                                                           |
| no_matches_files.log                | 1.5 KB | Log of raw files that produced no regex matches during parsing (21 files)                                   |
| index.html                          | 7.4 KB | Original WoWAH dataset documentation page                                                                   |

## Pipeline Overview

The pipeline follows a strict **temporal cross-validation** design to prevent data leakage:

- **Training set:** all historical data up to month  $M$
- **Test set:** data from month  $M+1$  only
- **Prediction target:** whether the player will be active in month  $M+2$

This means features are built from data two months prior to the prediction target, giving one month of “intervention time” between when the model scores a player and when churn would occur.

---

## Scripts

### `parser_mp.py` — Raw Log Parsing

Reads the thousands of raw `.txt` log files from **WoWAH/** using a multiprocessing `ProcessPoolExecutor`. A regex pattern extracts the 7 relevant fields (timestamp, avatar ID, guild, level, race, class, zone) from each line and writes them to a consolidated CSV. Files with no regex matches are logged to `no_matches_files.log`.

### `convert_to_parquet.py` — CSV to Parquet

Reads the parsed CSV and converts it to a Parquet file with proper types (datetime parsing, integer fields). This is a one-time preprocessing step to speed up all downstream reads.

### `add_churn.py` — Churn Label Construction

Constructs the supervised learning target. The logic:

1. Generates a complete cross-product of all (character, month) pairs across the full date range.
2. Marks each pair `has_played = True` if the character has any heartbeat in that month, `False` otherwise.
3. Removes rows where the month precedes a character's first-ever observed month (those would be unrealistic “never existed yet” negatives).
4. Saves the result to `wowah_churn_data.parquet`.

### `adding_features.py` — Feature Engineering

The core feature library. Features fall into three categories:

**Session features** (via `skrub.SessionEncoder`): Groups consecutive heartbeats into sessions using a configurable inactivity gap (30 or 60 minutes). From sessions computes:

- Monthly session count
- Total session duration per month
- Average session duration per month

**Player features** (monthly, per character):

- Max level reached
- Number of unique zones visited
- Most frequent zone
- Number of guilds joined / first and last guild

**Class-level features** (monthly aggregates across the whole playerbase by class):

- Average level for the player's class

- Number of active players for the player's class

**Location features** (`add_location_features`):

- **Zone rarity**:  $\log(N / n_{\text{visitors}})$  — how niche a zone is relative to the total playerbase. High rarity = few players visit it.
- **Hub identification**: zones below the 10th percentile of rarity are flagged as hubs (very popular zones).
- **Player rarity score**: max and mean rarity of all zones a player visited; fraction of time spent in hubs.
- **Location Gini coefficient**: measures how evenly a player distributes their time across zones (excluding hubs). Low Gini = grinding one spot; high Gini = exploring broadly.

## **main.py — ML Pipeline and Cross-Validation**

Orchestrates everything into a `skrubs` data operation graph:

1. Loads `wowah_churn_data.parquet` (optionally sampling 10% of users for speed).
2. Defines the `Splitter` class: a custom scikit-learn-compatible cross-validator that iterates month by month, always using all past months as training data and one future month as the test set.
3. Calls `add_features()` which loops over each unique month in the query set, filters historical data to the appropriate two-month offset, runs session encoding, and builds all features — ensuring zero leakage.
4. Vectorizes features with `skrubs.TableVectorizer`.
5. Fits a `HistGradientBoostingClassifier` (gradient boosted trees with native support for missing values).

Hyperparameters exposed for search:

- `session_gap`: 30 or 60 minutes
- `use_location`: whether to include location features (slower)

The entry point `cross_validate()` runs the full temporal cross-validation and prints results.

## **exploration.py / exploring\_data.py — Data Exploration**

Notebook-style scripts (using `# %%` cell markers) for initial data exploration: - `exploration.py`: generates a `skrubs.TableReport`, plots average level and character count by month on a dual-axis chart. - `exploring_data.py`: implements and compares Polars vs. Pandas sessionization approaches on the raw data (using a 20-minute inactivity gap for session splitting).

## **location\_features.py — Location Feature Exploration**

A standalone exploration script for location-based behavioral signals. Goes beyond what ends up in the final pipeline, experimenting with:

- Level-adjusted zone offsets (is a player over- or under-leveled for the zones they frequent?)
- **Temporal variance**: standard deviation and entropy of session start/end hours (in circular/radian space to handle midnight wrap-around) and day-of-week — low variance suggests habitual play patterns.
- **Spatial entropy**: normalized entropy of time-per-zone distribution (filtering out hubs and very short visits) — low entropy means the player focuses on few locations.
- Fraction of playtime in top-3 zones (with and without hubs).
- Exploratory HDBSCAN clustering of players on Gini  $\times$  spatial entropy.

### `cluster_users.py` — User Clustering

Uses the `skrub` data operation API to build a clustering pipeline:

1. Applies `SessionEncoder` to produce sessions.
2. Engineers session-level, character-level, and playerbase-level features.
3. Vectorizes with `TableVectorizer` + `DatetimeEncoder` (circular periodic encoding).
4. Aggregates all features to one row per user (mean).
5. Scales with `skrub.SquashingScaler`, imputes missing values.
6. Runs HDBSCAN to identify player archetypes.
7. Visualizes clusters with PCA (2D) and scatter plots.

### `plot_session_start_end.py` — Session Timing Visualization

Produces **polar clock plots** showing the distribution of session start and end times across all players in 15-minute bins. The polar projection naturally handles the circular nature of time-of-day (midnight wrap-around). Useful for understanding when players tend to log in and log out.

### `src/utils.py` — Utility Functions

- `sample_by_user(df, fraction)`: samples a fraction of unique characters and returns all their rows (preserves complete trajectories, not random row samples).
- `get_session_duration(df)`: computes session start, end, and duration from a session-encoded dataframe; floors single-heartbeat sessions at 1 minute to avoid zero-duration sessions masking activity.

---

## Technology Stack

| Library                                        | Role                                                                                                                                  |
|------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| <code>polars</code>                            | Fast dataframe operations throughout (lazy evaluation, Parquet I/O)                                                                   |
| <code>skrub</code>                             | <code>SessionEncoder</code> , <code>TableVectorizer</code> , <code>DatetimeEncoder</code> , data operation graph ( <code>skb</code> ) |
| <code>scikit-learn</code>                      | <code>HistGradientBoostingClassifier</code> , <code>HDBSCAN</code> , <code>KMeans</code> , <code>PCA</code> , cross-validation        |
| <code>matplotlib</code> / <code>seaborn</code> | Visualization                                                                                                                         |
| <code>concurrent.futures</code>                | Multiprocessing for raw log parsing                                                                                                   |

---

## Summary

In short: this repo ingests a large public dataset of World of Warcraft player activity logs, parses and cleans the raw files, engineers behavioral features at the session and monthly level (how much players play, where they go, how they distribute their time), and trains a gradient-boosted classifier to predict one month in advance which players are about to quit. The temporal cross-validation strategy is carefully designed so that no future information ever leaks into training features or labels.